

Assignment 2

Supervised Learning
(CNN Architecture, Optimizers, Transfer Learning)

Course: Supervised Learning

Faculty: Computer and Artificial Intelligence

University: Cairo University, Egypt

Dataset: CIFAR-10 (`keras.datasets.cifar10`)

Submission: Jupyter Notebook (.ipynb) — all cells executed

Important Rules

- Do NOT change the random seeds or the train/val/test split provided in the starter code.
- Every experiment must include: working code, results table, and required plots.
- Use labeled axes, titles, and legends on ALL plots.
- Use loops or functions to avoid copy-pasting — write clean, reusable code.
- Add brief comments explaining what each code block does.
- Name your submission file: StudentID_CNN_Assignment.ipynb
- All cells must be executed in order before submission.

Starter Code

Copy the following code into the **FIRST cell** of your notebook. Do not modify it.

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, models, callbacks
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.utils import to_categorical
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns
import time

# Reproducibility
np.random.seed(42)
tf.random.set_seed(42)

# CIFAR-10 Class Names
CLASS_NAMES = ['airplane', 'automobile', 'bird', 'cat', 'deer',
                'dog', 'frog', 'horse', 'ship', 'truck']

# Load Data
(x_train_full, y_train_full), (x_test, y_test) = cifar10.load_data()

# Fixed Split: 40,000 train / 10,000 val / 10,000 test
x_train = x_train_full[:40000].astype('float32')
y_train = y_train_full[:40000]
x_val = x_train_full[40000:].astype('float32')
y_val = y_train_full[40000:]
x_test = x_test.astype('float32')

print(f"Train: {x_train.shape} | Val: {x_val.shape} | Test: {x_test.shape}")
print(f"Pixel range: [{x_train.min()}, {x_train.max()}]")
print(f"Image shape: {x_train[0].shape}")
```

Helper Function (use this throughout your notebook):

```

def train_and_evaluate(model, x_tr, y_tr, x_v, y_v, x_te, y_te,
                       epochs=20, batch_size=128, extra_callbacks=None):
    cb = extra_callbacks if extra_callbacks else []
    start = time.time()
    history = model.fit(x_tr, to_categorical(y_tr, 10),
                        validation_data=(x_v, to_categorical(y_v, 10)),
                        epochs=epochs, batch_size=batch_size,
                        callbacks=cb, verbose=0)
    elapsed = time.time() - start
    test_loss, test_acc = model.evaluate(x_te, to_categorical(y_te, 10), verbose=0)
    print(f"Test Acc: {test_acc:.4f} | Test Loss: {test_loss:.4f} | Time:
{elapsed:.1f}s")
    return history, test_acc, test_loss, elapsed

def plot_curves(histories, labels, metric='val_accuracy', title='', ylabel=''):
    plt.figure(figsize=(10, 6))
    for h, lbl in zip(histories, labels):
        plt.plot(h.history[metric], label=lbl)
    plt.xlabel('Epoch')
    plt.ylabel(ylabel if ylabel else metric)
    plt.title(title)
    plt.legend()
    plt.grid(True)
    plt.show()

```

Task 1: Data Preprocessing Experiments

1A. Normalization Comparison

Build the following CNN architecture (call it "BaselineCNN"):

```
Conv2D(32, 3×3, ReLU, padding='same') → MaxPooling2D(2×2) →  
Conv2D(64, 3×3, ReLU, padding='same') → MaxPooling2D(2×2) →  
Flatten → Dense(128, ReLU) → Dense(10, Softmax)
```

Train this **same architecture** 3 times with different input preprocessing:

Experiment	Preprocessing
Exp A	No normalization (raw pixel values 0–255)
Exp B	Min-Max normalization: $x / 255.0 \rightarrow \text{range } [0, 1]$
Exp C	Standardization per-channel: $x' = (x - \text{mean}) / \text{std}$ (compute mean & std from training set ONLY)

Training config: 20 epochs, batch_size=128, Adam(learning_rate=0.001)

Deliverables:

1. Code for all 3 experiments
2. Plot: training loss curves — all 3 on ONE graph
3. Plot: validation accuracy curves — all 3 on ONE graph
4. Fill in the results table:

Preprocessing	Train Acc	Val Acc	Test Acc	Loss @ Epoch 1
None (0–255)				
Min-Max [0,1]				
Standardized				

1B. Data Augmentation Comparison

Using the **standardized** data from Exp C, train the BaselineCNN with and without data augmentation.

Augmentation configuration:

```
datagen = tf.keras.preprocessing.image.ImageDataGenerator(  
    rotation_range=15,  
    width_shift_range=0.1,  
    height_shift_range=0.1,  
    horizontal_flip=True,  
    zoom_range=0.1  
)  
datagen.fit(x_train_standardized)
```

Training config: 40 epochs, batch_size=128, Adam(lr=0.001)

Deliverables:

5. Code for both experiments

6. Single plot with 4 curves: train_acc (no aug), val_acc (no aug), train_acc (with aug), val_acc (with aug)
7. Report the gap (train_acc – val_acc) at epoch 40 for each. Which one overfits less?

Task 2: CNN Architecture Experiments

Use standardized data for ALL experiments in this task.

Training config (unless stated otherwise): Adam(lr=0.001), 20 epochs, batch_size=128

2A. Filter Count Comparison

Architecture (only filter counts change):

```
Conv2D(f1, 3, relu, same) → Conv2D(f2, 3, relu, same) → MaxPool(2,2) →  
Conv2D(f3, 3, relu, same) → Conv2D(f4, 3, relu, same) → MaxPool(2,2) →  
Flatten → Dense(256, relu) → Dense(10, softmax)
```

Build and train 3 models with different filter counts:

Model	f1	f2	f3	f4
Small	8	8	16	16
Medium	32	32	64	64
Large	64	64	128	128

Deliverables:

- Code for all 3 models (use a function that takes filter sizes as parameters)
- Print model.summary() for each — note total parameter count
- Fill in the results table:

Model	Total Params	Train Acc	Val Acc	Test Acc	Time (s)
Small					
Medium					
Large					

- Plot: val accuracy curves for all 3 on one graph

2C. Network Depth Comparison

All use 32 filters, kernel 3×3, ReLU, padding='same'. Build 3 models with different depths:

Shallow (4 conv layers):

```
Conv(32) → Conv(32) → MaxPool(2,2) →  
Conv(32) → Conv(32) → MaxPool(2,2) →  
Flatten → Dense(256, relu) → Dense(10, softmax)
```

Medium (6 conv layers):

```
Conv(32) → Conv(32) → MaxPool(2,2) →  
Conv(32) → Conv(32) → MaxPool(2,2) →  
Conv(32) → Conv(32) → GlobalAveragePooling2D →  
Dense(256, relu) → Dense(10, softmax)
```

Deep (8 conv layers):

```
Conv(32) → Conv(32) → MaxPool(2,2) →  
Conv(32) → Conv(32) → MaxPool(2,2) →  
Conv(32) → Conv(32) → MaxPool(2,2) →  
Conv(32) → Conv(32) → GlobalAveragePooling2D →  
Dense(256, relu) → Dense(10, softmax)
```

Deliverables:

12. Code for all 3 models
13. Results table (same columns as 2A)
14. For each model, plot training loss + val loss (2 curves per plot, 3 separate plots)
15. Does the deepest model perform best? Why or why not?

Task 3: Regularization Experiments

Use the Medium model from Task 2A as the baseline for all experiments.

3A. Dropout Rate Comparison

Train 3 models with different dropout configurations:

Model	Dropout Placement	Rate
D0	No dropout	0.0
D1	After each MaxPool + before output Dense	0.25
D2	After each MaxPool + before output Dense	0.5

Training config: 30 epochs, Adam(lr=0.001), batch_size=128

Deliverables:

- Code for all 3 models
- Compute overfitting gap = (train_acc – val_acc) at the last epoch
- Fill in the results table:

Model	Train Acc	Val Acc	Test Acc	Overfit Gap
D0				
D1				
D2				

- Plot: all 3 val accuracy curves on one graph
- Plot: all 3 train accuracy curves on one graph
- Which config gives the best test accuracy? Which gives the smallest overfit gap?

3B. Early Stopping Comparison

Using model D0 (no dropout), train with different early stopping configurations:

Experiment	Early Stopping Config
ES0	No early stopping — run for 50 epochs
ES2	monitor='val_loss', patience=5
ES3	monitor='val_loss', patience=10

Use `restore_best_weights=True` for all early stopping experiments.

```
# Example for ES2:  
es = callbacks.EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
```

Deliverables:

- Code for all 3 experiments
- Fill in the results table:

Experiment	Stopped at Epoch	Best Val Loss	Test Acc	Time (s)
ES0	50			
ES2				
ES3				

24. Plot: val loss for ES0, ES2, ES3 on one graph (mark the stopping point with a vertical line)

25. Is patience=5 too aggressive? Is patience=10 too lenient? Justify from your results.

Task 4: Optimizer Comparison

Use the Medium model from Task 2A. Use standardized data.

4A. Same Learning Rate — 5 Optimizers

Train with all 5 optimizers at learning_rate = 0.001, 30 epochs, batch_size=128:

```
optimizers = {  
    'SGD': keras.optimizers.SGD(learning_rate=0.001),  
    'Momentum': keras.optimizers.SGD(learning_rate=0.001, momentum=0.9),  
    'AdaGrad': keras.optimizers.Adagrad(learning_rate=0.001),  
    'RMSProp': keras.optimizers.RMSprop(learning_rate=0.001),  
    'Adam': keras.optimizers.Adam(learning_rate=0.001),  
}
```

Deliverables:

26. Code for all 5 runs (use a loop)
27. Fill in the results table:

Optimizer	Train Loss (final)	Val Loss (final)	Test Acc	Time (s)
SGD				
Momentum				
AdaGrad				
RMSProp				
Adam				

28. Plot: all 5 training loss curves on one graph
29. Plot: all 5 val accuracy curves on one graph
30. Which optimizer converges fastest (reaches val_acc > 0.65 first)?

4B. Learning Rate Sensitivity — Adam

Using the Adam optimizer, train with 3 different learning rates (30 epochs, batch_size=128):

Experiment	Learning Rate
LR1	0.0001
LR2	0.001
LR3	0.01

Deliverables:

31. Code for all 3 runs
32. Fill in the results table:

Learning Rate	Train Loss (final)	Val Loss (final)	Test Acc
0.0001			
0.001			
0.01			

33. Plot: all 3 training loss curves on one graph
34. Plot: all 3 val accuracy curves on one graph
35. Which learning rate is too slow? Which is unstable? Which is the best?

Task 5: Performance Evaluation

5A. Best Model Selection & Evaluation

Based on ALL your experiments, choose the best combination of:

- Preprocessing method
- Architecture (filters, kernel size, depth)
- Dropout configuration
- Optimizer and learning rate
- Batch size

Train your best model for **50 epochs** with early stopping (patience=7, restore_best_weights=True) and data augmentation.

Deliverables:

36. State which settings you chose and why (1–2 sentences each)
37. Report the final test accuracy
38. Display the confusion matrix as a heatmap (label axes with class names):

```
y_pred = np.argmax(model.predict(x_test_processed), axis=1)
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=CLASS_NAMES, yticklabels=CLASS_NAMES)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()

print(classification_report(y_test, y_pred, target_names=CLASS_NAMES))
```

5B. Error Analysis

Using your best model's predictions on the test set:

39. Find the 3 worst-performing classes (lowest recall).
40. For each of those 3 classes, display 5 misclassified images with their true label and predicted label.
41. Find the top 3 most confused class pairs (e.g., cat↔dog). Display 4 examples from each confusion direction.
42. Why are these classes hard to distinguish? (Think about visual similarity at 32×32 resolution.)

Helper code:

```
misclassified_idx = np.where(y_pred != y_test.flatten())[0]
print(f"Total misclassified: {len(misclassified_idx)} / {len(y_test)}")

def show_misclassified(true_class, n=5):
    idx = [i for i in misclassified_idx if y_test[i] == true_class][:n]
    fig, axes = plt.subplots(1, n, figsize=(15, 3))
    for ax, i in zip(axes, idx):
        ax.imshow(x_test[i].astype('uint8'))
        ax.set_title(f"True: {CLASS_NAMES[y_test[i][0]]}\nPred:
{CLASS_NAMES[y_pred[i]]}")
        ax.axis('off')
    plt.suptitle(f"Misclassified '{CLASS_NAMES[true_class]}'")
    plt.tight_layout()
    plt.show()
```

Task 6: Transfer Learning

6A. Feature Extraction vs Fine-Tuning vs From Scratch

Step 1 — Resize CIFAR-10 for VGG16:

```
x_train_resized = tf.image.resize(x_train_standardized, (48, 48)).numpy()  
x_val_resized   = tf.image.resize(x_val_standardized,   (48, 48)).numpy()  
x_test_resized  = tf.image.resize(x_test_standardized,  (48, 48)).numpy()
```

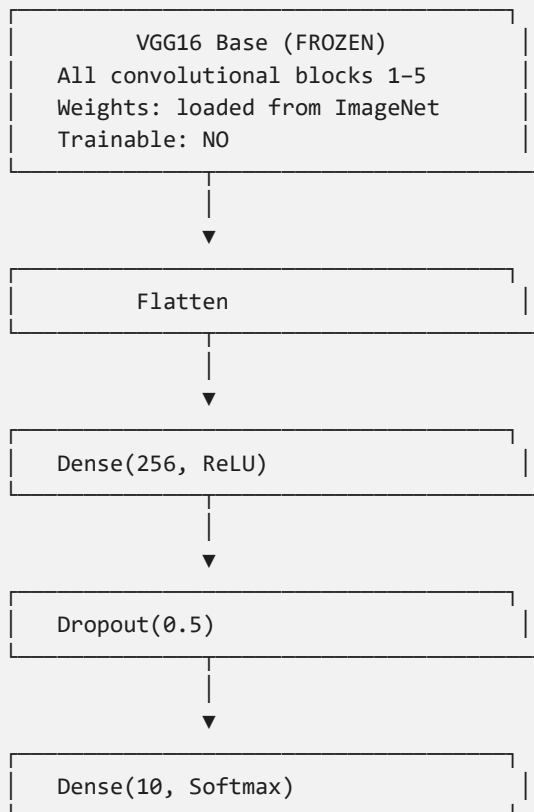
Model 1: From Scratch

Build the Medium CNN from Task 2A with input_shape=(48, 48, 3). Train 20 epochs, Adam(lr=0.001), batch_size=128.

Model 2 — Feature Extraction (Frozen VGG16)

- Load a pre-trained VGG16 model (trained on ImageNet) without the top classification layers.
- Freeze ALL VGG16 layers so their weights do not update during training.
- Add your own classification head on top.

Architecture Diagram:

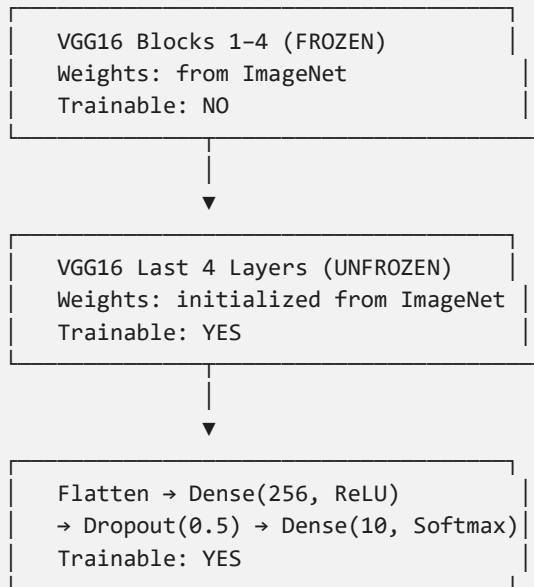


Compile with: Adam(lr=0.001), categorical_crossentropy. Train 20 epochs, batch_size=128.

Model 3 — Fine-Tuning (Partial Unfreeze)

- Load a pre-trained VGG16 model (same as Model 2).
- Freeze all layers EXCEPT the last 4 layers — these will be re-trained on CIFAR-10.
- Add the same classification head (Flatten → Dense(256) → Dropout → Dense(10)).
- Use a LOWER learning rate (1e-5) to avoid destroying the pre-trained weights.

Architecture Diagram:



Compile with: `Adam(lr=1e-5)`, `categorical_crossentropy`. Train 20 epochs, `batch_size=128`.

Same architecture and frozen layers as Model 3, but use `lr=0.001` instead of 1e-5. Train 20 epochs.

Model 4 — Fine-Tuning (Higher Learning Rate)

Same architecture and frozen layers as Model 3 (unfreeze last 4 VGG16 layers), but use a **higher learning rate (lr=0.001)** instead of 1e-5. This tests whether a high learning rate destroys the pre-trained weights. Train 20 epochs, `batch_size=128`.

Deliverables:

43. Code for all 4 models
44. Print `model.summary()` for Models 2 and 3 — note trainable vs non-trainable params
45. Fill in the results table:

Model	Trainable Params	Non-Trainable Params	Test Acc	Time (s)
From Scratch		0		
Feature Extraction				

Fine-Tune (lr=1e-5)				
Fine-Tune (lr=1e-3)				

46. Plot: val accuracy curves — all 4 models on one graph
47. Plot: val loss curves — all 4 models on one graph
48. Why does Model 4 (lr=0.001) perform differently from Model 3 (lr=1e-5)?
49. Does transfer learning outperform training from scratch on CIFAR-10? Why or why not?

Experiments Checklist

Use this checklist to ensure you have completed all parts before submission:

#	Experiment	Models/Runs	Done?
1A	Normalization comparison	3	☐
1B	Data augmentation comparison	2	☐
2A	Filter count comparison	3	☐
2C	Network depth comparison	3	☐
3A	Dropout rate comparison	3	☐
3B	Early stopping comparison	3	☐
4A	Optimizer comparison	5	☐
4B	Learning rate comparison	3	☐
5A	Best model + confusion matrix	1	☐
5B	Error analysis	—	☐
6A	Transfer learning	4	☐
	TOTAL MODEL TRAININGS	~27	

Team Members Contribution

Each team must fill in the table below. **Be specific** about who did what. An example is provided in the first row.

Member Name	Tasks Completed	Specific Contributions	Time Spent (approx.)
Ahmed Mohamed (ID: 20231001)	Task 1A, 1B Task 3A	- Implemented all 3 normalization methods - Plotted loss & accuracy curves for Task 1A - Built all 3 dropout models (D0, D1, D2) - Wrote analysis for overfitting comparison	~8 hours
Sara Ali (ID: 20231002)	Task 2A, 2B Task 4A	- Built CNN models with different filter counts (Small, Medium, Large) - Built depth comparison models (Shallow, Medium, Deep) - Ran all 5 optimizer experiments in a loop - Created comparison plots for Tasks 2A, 4A	~10 hours
Omar Hassan (ID: 20231003)	Task 5A, 5B Task 6	Selected best model and trained with early	~12 hours

		<i>stopping + augmentation</i> • <i>Generated confusion matrix & classification report</i> • <i>Performed error analysis (misclassified images)</i> • <i>Implemented all 4 transfer learning models</i> • <i>Wrote final analysis for transfer learning</i>	

Note: The example rows above (in gray italic) are for demonstration only. Delete them and fill in your actual team contributions. Add more rows if your team has more than 3 members.

— Good Luck! —